

OpenFastVC: An Open Source Library for Video Coding Fast Algorithm Implementation

Hang Yuan

hyuan@pku.edu.cn

Peking University Shenzhen Graduate School
Peng Cheng Laboratory
China

Wei Gao*

gaowei262@pku.edu.cn

Peking University Shenzhen Graduate School
China

ABSTRACT

Despite the remarkable coding gains exhibited by the recently released new-generation video coding standards, their serious computational complexity will pose a significant challenge in coding latency to practical applications. Therefore, the corresponding low-complexity optimizations assume paramount importance. To facilitate the research in this field, the first open source software library for video coding fast algorithm implementation, namely OpenFastVC, is proposed in this paper. Specifically, OpenFastVC offers the outputting and processing of the intermediate coding information, e.g., the CU partitioning results, which is indispensable to fast algorithm design. To facilitate the integration of the designed algorithms, OpenFastVC also provides application programming interfaces (APIs) for direct control over the encoding process. Moreover, the existing typical fast algorithms are further implemented in OpenFastVC, enabling researchers to evaluate the performance of their algorithm effortlessly. The release of this library is highly favorable for the design, implementation, and evaluation of video coding fast algorithms, thereby fostering the widespread adoption of the new coding standards. The open source library for OpenFastVC is available at <https://openi.pcl.ac.cn/OpenCompression/OpenFastVC>.

CCS CONCEPTS

• **Computing methodologies** → **Image compression**; • **Software and its engineering** → **Open source model**.

KEYWORDS

video coding, low-complexity optimization, fast algorithm, open source software

ACM Reference Format:

Hang Yuan and Wei Gao. 2023. OpenFastVC: An Open Source Library for Video Coding Fast Algorithm Implementation. In *Proceedings of the*

*Corresponding author. This work was supported by Natural Science Foundation of China (62271013, 62031013), Shenzhen Fundamental Research Program (GXWD20201231165807007-20200806163656003), Shenzhen Science and Technology Plan Basic Research Project (JCYJ20190808161805519), and was sponsored by CAAI-Huawei MindSpore Open Fund (CAAIJSJLJJ-2022-002C).

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

MM '23, October 29–November 3, 2023, Ottawa, ON, Canada

© 2023 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-0108-5/23/10...\$15.00

<https://doi.org/10.1145/3581783.3613465>

31st ACM International Conference on Multimedia (MM '23), October 29–November 3, 2023, Ottawa, ON, Canada. ACM, New York, NY, USA, 4 pages.
<https://doi.org/10.1145/3581783.3613465>

1 INTRODUCTION

In response to the surging demand for video compression, new-generation coding standards with excellent performance have been released by various international organizations recently [6, 9, 16]. The application of these standards is poised to substantially reduce video transmission costs. However, the improvement in coding gains primarily stem from more refined coding tools, which also obviously increase the computational complexity [7, 8, 13, 15]. The resulting coding latency undoubtedly is a significant barrier in the promotion and application of new standards. There is no doubt that the low-complexity optimization represents a pivotal research topic for the development of coding standards. Nevertheless, compared with the extensive and diverse researches on fast algorithms in previous coding standards, there remains a relative dearth of accelerating schemes for current new standards. Consequently, accelerating the research progress of low complexity optimization and promoting the prosperity of this field holds significant implications for the promotion of new coding standards.

After over three decades of development, video coding technology has become highly intricate [5, 10–12]. Although there are many documents, blogs, and papers available, most of them only explain the principles or workflows corresponding to the coding standards and tools. It is still extremely difficult for a researcher to clearly understand the implementation of a new coding module at the code-level and design the corresponding optimization scheme. Furthermore, the reference software of coding standards is commonly implemented in C/C++ to ensure execution efficiency. Compared with employed the straightforward coding languages, e.g., Python, this choice adds an additional layer of difficulty for researchers in the implementation and evaluation of their proposed methods. Hence, the development and integration speed of low-complexity algorithms for new coding standards are severely hampered.

In order to address this challenge, the first fast algorithm implementation-oriented open source library for video coding, named OpenFastVC, is developed for new coding standards. The main contributions of OpenFastVC are as follows. **(1) Rapid acquisition of intermediate coding information:** The design of fast algorithm generally is based on the statistics and analysis of intermediate coding information generated during the execution of the target coding module, while these information is not readily available in the reference software. To facilitate the analysis, OpenFastVC implements

the macro switch on original coding platform, which furnishes the requisite information directly, thereby economizing researchers' time. (2) **Efficient integration of optimization schemes:** Seamlessly integrating the devised fast algorithm into original encoder requires researchers to have an in-depth comprehension of the logical structure within the code, which is time-consuming. OpenFastVC supplies the application programming interfaces (APIs), where low-complexity optimizations, e.g., early termination and mode pruning, can be directly achieved by inputting the result of proposed algorithm in a preset format. (3) **Effortless evaluation of coding performance:** After completing the integration of the designed algorithm, evaluating the coding performance emerges as an indispensable stride. OpenFastVC facilitates this endeavor by proffering the implementation of typical algorithms, thereby affording researchers a convenient and expeditious means to finalize their algorithms.

2 APPLICATION IN MAINSTREAM CODING STANDARDS

Video coding primarily consists of five stages: coding unit (CU) partition, prediction, transformation, quantification, and entropy coding. Among them, CU partition is the most time-consuming stage [14]. Thus, the current version OpenFastVC is mainly aimed at the low-complexity optimization of this stage in mainstream coding standards, i.e., the third-generation audio video coding standard (AVS3), versatile video coding (VVC), and video-based point cloud compression (V-PCC) [5, 9, 12]. Besides, the intra prediction in VVC is also supported. Note that OpenFastVC is developed based on the reference software of these coding standards [1–3]. Although the reference software may undergo version updates, the underlying code structure generally remains unchanged, which does not hinder the utilization of OpenFastVC.

2.1 Application in AVS3

AVS3 is the third-generation coding standard released by audio video coding standard workgroup of China [16]. Compared to its previous generation standard AVS2, over 30% coding gains have been achieved, while the price is the significant escalation in computational complexity, especially in CU partition. As illustrated in Figure 1, the CU partitioning modes in AVS3 have increased from 2 to 6, including Non-Split, Quad-Tree (QT), Vertical Binary Tree (Ver BT), Horizontal BT (Hor BT), Vertical Extended QT (Ver EQT), and Horizontal EQT (Hor EQT). Additionally, the partitioning depth has also expanded from $\{8 \times 8$ to $64 \times 64\}$ to $\{4 \times 4$ to $128 \times 128\}$. Since the optimal mode selection utilizes the brute-force approach to try all the options and the CU split is a recursive process (where the larger CU cannot be determined until smaller CUs have completed their decisions), the coding latency in CU partition is quite severe.

The proposed OpenFastVC is meticulously devised to provide convenience for the research to accelerate the coding stage. Specifically, the CU partitioning information can be output in coding log during the encoding process by activating the corresponding macro switch (setting *OFVC_OUTPUT_CU_INFO* to 1). An example of the outputted partitioning information is presented in Listing 1. x and y are the position of current CU. w and h denote the CU width and height, respectively. qt_depth reflects how many times

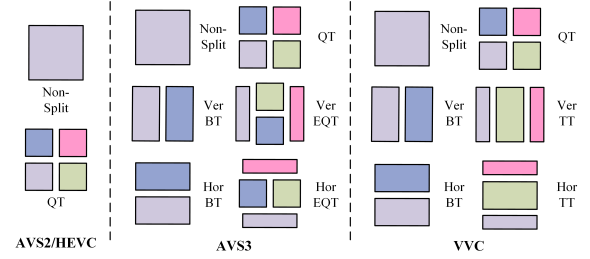


Figure 1: The expansion in CU partitioning modes.

the current CU has undergone QT partition thus far. *bet_depth* denotes the split times by BT and EQT modes. *split_mode* is the optimal split mode of current CU (0: Non-Split, 1: Ver BT, 2: Hor BT, 3: Ver EQT, 4: Hor EQT, and 5: QT). Based on this information and the original video sequence, researchers can effortlessly analyze the partitioning result and further design the optimization scheme.

Listing 1: An example of the outputted CU partitioning information in AVS3.

```
1 x:0-y:0-w:64-h:64-qt_depth:1-bet_depth:0-split_mode:1
2 x:0-y:0-w:32-h:64-qt_depth:1-bet_depth:1-split_mode:0
```

Based on OpenFastVC, implementing the designed fast algorithms into the encoder is also highly convenient. The CU partition can be controlled by activating the macro switch (setting *OPFV_CU_DECISION_CTRL* to 1) and assigning the result of the proposed algorithm to *fast_split_results*[6]. Note that there are six digits in *fast_split_results*[6], which sequentially control the usage of modes Non Split, Ver BT, Hor BT, Ver EQT, Hor EQT, and QT. The value 0 represents removing the corresponding partitioning mode from candidate list, and 1 denotes retaining this mode.

In order to facilitate the performance evaluation of the designed fast algorithm and provide a reference for controlling CU partitioning decision, we have implemented the low-complexity optimization proposed by [15] in OpenFastVC. To be specific, [15] finds that the refined partitioning modes, newly added in AVS3, are quite sensitive to texture changes. Therefore, the usage of EQT and BT is restricted in areas with minimal texture variation to accelerate the CU decision. This algorithm can be enabled by setting *OPFV_CU_DECISION_REF_ALG1* to 1.

2.2 Application in VVC

Similar to AVS3, the computational complexity in VVC is also severe. Its partitioning depth has similarly reached $\{4 \times 4$ to $128 \times 128\}$, and the partitioning modes has also expanded from 2 to 6, where one notable difference is that VVC utilizes the Trinary Tree (TT) mode rather than EQT in AVS3, as shown in Figure 1. Moreover, more encoding tools have also been incorporated into intra prediction to achieve better gains, e.g., the expanded prediction modes, multiple reference lines, and intra sub-partition (ISP), which further exacerbates the coding latency. Therefore, the current version of OpenFastVC primarily focuses on supporting the CU partition and intra prediction of VVC.

The CU partitioning information of VVC can be output after turning on the macro switch (setting *OFVC_OUTPUT_CU_INFO* to 1). Note that the reference software of VVC only records the final optimal partitioning result. Hence, a Python script (*PartitioningInfoE*

xtract.py) is additionally provided in OpenFastVC to obtain intermediate information during CU decision, e.g., the split depth and best partitioning mode. Listing 2 and Listing 3 depict the original output and the processed result, respectively. The definitions of x , y , w , h , and qt_depth are same as Listing 1. However, different from AVS3, *split_mode*:3 or *split_mode*:4 denotes that the optimal split mode of the current CU is Ver TT or Hor TT. In addition, *bt_depth* indicates the BT partitioning times of the current CU. *mt_depth* is the partitioning times by BT and TT. Luma and Chroma CUs are independently partitioned in VVC, thus *LUMA* and *CHROMA* represent the CU type. Besides, enabling the macro switch *OFVC_OUTPUT_IP_INFO* will activate the output of intra prediction information, and an example is shown in Listing 4. All the intra prediction modes in the candidate list are output, where *MID* is the ID of each mode, and *ISPF* is the ISP flag (-1: disabling ISP; 0: horizontal ISP; 1: vertical ISP). Moreover, *BestMode* indicates the optimal intra prediction mode of the current CU.

Listing 2: An example of the original partition information that has been outputted in VVC.

```
1 LUMA : [x:0-y:0-w:32-h:16]&&[depth:3-qt_depth:2-bt_depth:1-mt_depth:1] =>
  ParentCU[x:0-y:0-w:32-h:32] => ParentCU[x:0-y:0-w:64-h:64] =>
  ParentCU[x:0-y:0-w:128-h:128]
2 CHROMA : [x:0-y:0-w:16-h:16]&&[depth:2-qt_depth:2-bt_depth:0-mt_depth:0] =>
  ParentCU[x:0-y:0-w:32-h:32] => ParentCU[x:0-y:0-w:64-h:64]
```

Listing 3: An example of the processed partition information in VVC.

```
1 LUMA : x:0-y:0-w:16-h:32-qt_depth:2-bt_depth:1-mt_depth:1-split_mode:0
2 CHROMA : x:0-y:0-w:16-h:16-qt_depth:2-bt_depth:0-mt_depth:0-split_mode:0
```

Listing 4: An example of the outputted intra prediction information in VVC.

```
1 [x:0-y:0-w:8-h:8]<=====[MID:0, ISPF:-1]<=====[MID:1, ISPF:-1]<=====[MID:50,
  ISPF:-1]<=====[MID:0, ISPF:0]<=====[MID:0, ISPF:1]<=====[MID:50, ISPF
  :0]<=====[MID:50, ISPF:1]<=====[BestMode:[MID:0, ISPF:-1]
```

In OpenFastVC, the researchers can control the CU partitioning mode selection by enabling the macro switch *OPFV_CU_DECISION_CTRL* and assigning values to *fast_split_results*[6] based on the outcome of their designed fast algorithm. The six digits within *fast_split_results*[6] correspond to Non-Split, Ver BT, Hor BT, Ver TT, Hor TT, and QT, respectively. The value 0 indicates that this mode is disabled, while the value 1 represents enabling it. Besides, setting *OPFV_IP_DECISION_CTRL* to 1 can achieve the control over intra prediction mode selection. Specifically, the availability of the ISP mode is controlled by *fast_ISP_flag* (true or false). If the ID of an intra prediction mode is added to the *disabling_mode_array*, it will be removed from the candidate list.

OpenFastVC provides the reference algorithm for CU partition and intra prediction for VVC, respectively. The reference fast CU decision is the implementation of the low-complexity algorithm proposed in [13]. This algorithm primarily considers the gradient of the current CU and the texture difference between its possible sub-CUs to guide the partition. It can be enabled by setting *OPFV_CU_DECISION_REF_ALG1* to 1. In addition, the reference fast intra prediction is the optimization of the low-complexity ISP mode decision devised in [8]. The decision to enable or disable ISP is based on the texture variation within the current CU and the difference between the reference pixels and the distant area of the

Table 1: Coding performance of the provided reference algorithm for AVS3.

Sequence	BD-Rate	ΔT
Vidyo1	1.44 %	0.41
Vidyo3	1.33 %	0.38
Vidyo4	1.71 %	0.37
FourPeople	1.51 %	0.45
Johnny	1.71 %	0.37
KristenAndSara	2.10 %	0.40
Average	1.63 %	0.40

current CU after applying ISP. Note that this fast algorithm can be utilized by setting *OPFV_ISP_DECISION_REF_ALG1* to 1.

2.3 Application in V-PCC

V-PCC is proposed for point cloud compression, which projects 3D point clouds into 2D video sequences, i.e., occupancy, geometry, and attribute sequences, and compresses these sequences by the video encoder. V-PCC inherits the significant computational complexity from video coding. More seriously, there are three high-quality 2D projection sequences that require encoding, which exacerbates time consumption. Therefore, OpenFastVC is also developed to facilitate low-complexity optimization for V-PCC, with the current version primarily focusing on supporting the most complex coding stage: CU partition.

Listing 5: An example of the outputted CU partitioning information in V-PCC.

```
1 Coding CU ==> x:64-y:0-w:64-split_mode:1
2 Coding CU ==> x:64-y:0-w:32-split_mode:0
```

The output of CU partitioning information can be achieved by setting *OFVC_OUTPUT_CU_INFO* to 1 in OpenFastVC, where an example is given in Listing 5. Note that the 2D video encoder here is the reference software of HEVC in which all the CUs are square-shaped. The CU size can be accurately represented by its width alone. Besides, there are only 2 partitioning modes. *split_mode* : 0 or *split_mode* : 1 denotes that the best partitioning mode of current CU is Non_Split or QT. Moreover, after opening the control over CU partition (setting *OPFV_CU_DECISION_CTRL* to 1), the integration of the designed algorithm is also simple. Assigning 1 or 0 to *fast_split_flag* indicates executing the original CU decision strategy for the current CU or early terminating the current partition.

The reference fast CU partitioning algorithm for V-PCC provided by OpenFastVC is the improved implementation of the low-complexity scheme proposed in [14] with better generalization (less but more typical sequences for training) and performance. The cross-projection information is used to guide the CU partition, e.g., the occupancy and geometry information is utilized to assist the attribute CU partition. Additionally, the rate-distortion performance of each CU is considered during the decision mode training to improve the prediction accuracy of CUs that may lead to serious coding loss. This algorithm can be enabled by setting *OPFV_CU_DECISION_REF_ALG1* to 1.

3 EVALUATION

To facilitate researchers in efficiently estimating the performance of their designed algorithms, we also provide performance evaluation of the reference fast algorithms. It should be noted that

Table 2: Coding performance of the provided reference algorithm for V-PCC.

Point Cloud	Geom.BD-GeomRate		End-to-End BD-AttrRate			Geom.BD-TotalRate		End-to-End BD-TotalRate			ΔT
	D1	D2	Luma	Cb	Cr	D1	D2	Luma	Cb	Cr	
Loot	0.00%	0.00%	0.17%	-1.80%	-0.76%	0.02%	0.02%	0.13%	-1.36%	-0.75%	64.75%
Redandblack	0.00%	0.00%	0.14%	-0.39%	0.01%	0.03%	0.03%	0.11%	-0.32%	0.00%	62.09%
Soldier	0.00%	0.00%	0.04%	-0.07%	-0.31%	-0.02%	-0.02%	0.04%	-0.07%	-0.22%	43.11%
Queen	0.00%	0.00%	0.20%	-1.00%	-1.07%	-0.09%	-0.08%	0.17%	-0.76%	-0.81%	59.63%
Longdress	0.00%	0.00%	0.01%	-0.19%	-0.08%	0.03%	0.03%	0.00%	-0.16%	-0.07%	49.98%
Basketball_player	0.00%	0.00%	0.49%	-2.02%	-1.90%	0.03%	0.03%	0.35%	-1.53%	-1.46%	69.56%
Dancer	0.00%	0.00%	0.55%	-1.81%	-2.11%	0.06%	0.06%	0.40%	-1.38%	-1.63%	68.41%
Average	0.00%	0.00%	0.23%	-1.04%	-0.89%	0.01%	0.01%	0.17%	-0.80%	-0.71%	59.65%

Table 3: Coding performance of the provided reference algorithms for VVC.

Class	Sequence	FastPartition		FastISP	
		BD-Rate	ΔT	BD-Rate	ΔT
A	Tango2	0.78%	40.25%	0.00%	10.79%
	FoodMarket4	0.70%	41.25%	0.02%	10.76%
	Campfire	1.76%	54.00%	-0.01%	13.38%
	CatRobot	2.20%	49.20%	0.26%	10.34%
	DaylightRoad2	2.54%	58.30%	0.43%	12.20%
	ParkRunning3	0.47%	41.06%	0.07%	7.43%
B	MarketPlace	1.39%	58.20%	0.10%	13.30%
	RitualDance	2.08%	52.14%	0.29%	12.51%
	Cactus	2.33%	55.31%	0.45%	13.41%
	BasketballDrive	1.98%	57.96%	0.63%	12.08%
	BQTerrace	2.11%	46.89%	0.60%	13.13%
C	BasketballDrill	3.74%	41.56%	0.85%	14.57%
	BQMall	2.02%	41.90%	0.80%	10.97%
	PartyScene	1.11%	33.14%	0.50%	13.40%
	RaceHorses	1.09%	43.66%	0.35%	12.21%
D	BasketballPass	3.04%	57.36%	0.66%	11.33%
	BQSquare	2.47%	52.90%	0.71%	13.36%
	BlowingBubbles	1.77%	53.19%	0.61%	14.85%
	RaceHorses	2.00%	57.47%	0.40%	14.22%
E	FourPeople	1.62%	40.10%	0.64%	12.17%
	Johnny	2.63%	48.33%	0.55%	11.95%
	KristenAndSara	1.83%	42.66%	0.74%	12.21%
Average		1.89%	48.49%	0.44%	12.30%

the experiments conducted for each reference fast algorithm follow the test conditions described in the corresponding paper. The compressing performance is evaluated by BD-Rate [4]. The time-saving is reflected in ΔT : $\Delta T = (T_{Org} - T_{Ref})/T_{Org}$. T_{Org} and T_{Ref} are the coding time of the original encoder and the encoder that incorporated the corresponding reference fast algorithm, respectively. The performance of the reference fast algorithm for AVS3 and V-PCC is separately illustrated in Table 1 and Table 2, where 40% and 59.65% of coding time can be separately reduced. Table 3 gives the performance of the reference fast algorithms for VVC. FastPartition and FastISP denote the low-complexity optimization for CU partition and ISP decision, and the coding time is saved by 48.49% and 12.30%, respectively. Nevertheless, the coding losses of the provided reference algorithms can be ignored.

4 CONCLUSION

To facilitate research on fast algorithms for new-generation video coding standards and mitigate the impact of complexity on the promotion of these standards, we have developed the first fast algorithm implementation-oriented open source library, named OpenFastVC. In this library, researchers can swiftly obtain the intermediate coding information by activating the macro switch, thereby providing the convenience for the design of fast algorithms. In addition, OpenFastVC affords APIs for controlling the encoding process. The designed fast algorithm can be promptly deployed by invoking these interfaces. Besides, the implementation of typical

low-complexity optimization schemes is also offered in the proposed library. This enables researchers to easily and efficiently evaluate their algorithms. We believe that the proposed OpenFastVC can provide strong support for low-complexity research in new-generation video coding standards and effectively promote the application of these standards.

REFERENCES

- [1] 2023. *HPM-9.0 software repository*. ftp://47.93.196.121/Public/codec/video_codec/HPM
- [2] 2023. *Point Cloud Compression Category 2 reference software, TMC2-12.0*. <http://mpegx.int-evry.fr/software/MPEG/PCC/TM/mpeg-pcc-tmc2.git>
- [3] 2023. *VVC reference Model, VTM-19.2*. https://vcgit.hhi.fraunhofer.de/jvet/VVCSoftware_VTM
- [4] G. Bjontegaard. VCEG, Austin, TX, USA, Doc. VCEG-M33, 2018. Calculation of average PSNR differences between RD-curves. (VCEG, Austin, TX, USA, Doc. VCEG-M33, 2018).
- [5] Benjamin Bross, Jianle Chen, Jens-Rainer Ohm, Gary J. Sullivan, and Ye-Kui Wang. 2021. Developments in International Video Coding Standardization After AVC, With an Overview of Versatile Video Coding (VVC). *Proc. IEEE* 109, 9 (2021), 1463–1493. <https://doi.org/10.1109/JPROC.2020.3043399>
- [6] Benjamin Bross, Ye-Kui Wang, Yan Ye, Shan Liu, Jianle Chen, Gary J. Sullivan, and Jens-Rainer Ohm. 2021. Overview of the Versatile Video Coding (VVC) Standard and its Applications. *IEEE Transactions on Circuits and Systems for Video Technology* 31, 10 (2021), 3736–3764. <https://doi.org/10.1109/TCSVT.2021.3101953>
- [7] Zhanyuan Cai and Wei Gao. 2021. Efficient Fast Algorithm and Parallel Hardware Architecture for Intra Prediction of AVS3. In *2021 IEEE International Symposium on Circuits and Systems (ISCAS)*. 1–5. <https://doi.org/10.1109/ISCAS51556.2021.9401121>
- [8] Xinchao Dong, Liquan Shen, Mei Yu, and Hao Yang. 2022. Fast Intra Mode Decision Algorithm for Versatile Video Coding. *IEEE Transactions on Multimedia* 24 (2022), 400–414. <https://doi.org/10.1109/TMM.2021.3052348>
- [9] Euee S. Jang, Marius Preda, Khaled Mamou, Alexis M. Tourapis, Jungsun Kim, Danilo B. Graziosi, Sungryeul Rhyu, and Madhukar Budagavi. 2019. Video-Based Point-Cloud-Compression Standard in MPEG: From Evidence Collection to Committee Draft [Standards in a Nutshell]. *IEEE Signal Processing Magazine* 36, 3 (2019), 118–123. <https://doi.org/10.1109/MSP.2019.2900721>
- [10] Didier Le Gall. 1991. MPEG: A Video Compression Standard for Multimedia Applications. *Commun. ACM* 34, 4 (apr 1991), 46–58. <https://doi.org/10.1145/103085.103090>
- [11] Ming Liou. 1991. Overview of the P×64 Kbit/s Video Coding Standard. *Commun. ACM* 34, 4 (apr 1991), 59–63. <https://doi.org/10.1145/103085.103091>
- [12] Siwei Ma, Li Zhang, Shiqi Wang, Chuanmin Jia, Shanshe Wang, Tiejun Huang, Feng Wu, and Wen Gao. 2022. Evolution of AVS video coding standards: twenty years of innovation and development. *Science China Information Sciences* 65, 9 (aug 2022). <https://doi.org/10.1007/s11432-021-3461-9>
- [13] Hao Yang, Liquan Shen, Xinchao Dong, Qing Ding, Ping An, and Gangyi Jiang. 2020. Low-Complexity CTU Partition Structure Decision and Fast Intra Mode Decision for Versatile Video Coding. *IEEE Transactions on Circuits and Systems for Video Technology* 30, 6 (2020), 1668–1682. <https://doi.org/10.1109/TCSVT.2019.2904198>
- [14] Hang Yuan, Wei Gao, Ge Li, and Zhu Li. 2022. Rate-Distortion-Guided Learning Approach with Cross-Projection Information for V-PCC Fast CU Decision. In *Proceedings of the 30th ACM International Conference on Multimedia (Lisboa, Portugal) (MM '22)*. Association for Computing Machinery, New York, NY, USA, 3085–3093. <https://doi.org/10.1145/3503161.3548215>
- [15] Hang Yuan, Wei Gao, and Junle Wang. 2021. Dynamic Computational Resource Allocation for Fast Inter Frame Coding in Video Conferencing Applications. In *2021 IEEE International Conference on Multimedia and Expo (ICME)*. 1–6. <https://doi.org/10.1109/ICME51207.2021.9428275>
- [16] Jiaqi Zhang, Chuanmin Jia, Meng Lei, Shanshe Wang, Siwei Ma, and Wen Gao. 2019. Recent development of AVS video coding standard: AVS3. In *Picture Coding Symposium (PCS)*. IEEE, 1–5.